Senior Design Test Plan

Motivation:

Analysis of specifications and implementation from set of different perspectives.
Test cases are repeatable and verifiable.
Test suites can be constructed as tool to quickly validate overall system.

Terminology:

- *normal* testing: testing with expected inputs in normal operating conditions.
- *abnormal* testing: testing with exceptional inputs or conditions.
- *boundary* testing: testing focused on subdivisions of an input domain
- *blackbox* testing: testing based on the requirements specification.
- *whitebox* testing: testing based on knowledge of the implementation.
- *functional* testing: testing based on expected features.
- *performance* testing: testing based on expected performance criteria such as speed, power consumption, accuracy within tolerances, etc.
- *unit* testing: testing of individual components or modules.
- *integration* testing: testing of interfaces between components and

Project Teams are Encouraged to use Software Testing Tools

- ► Free software testing tools
- ► Graphical user interface testing
- ► Load testing tools
- ► Unit testing frameworks

Test Plan

I. *Overall Test Plan*: at least one paragraph describing general testing strategies employed by your project.

II. *Test Case Description*:

1. test case identifier (a number or unique name)
2. purpose of test
3. description of test
4. inputs
5. expected outputs/results

---

6. normal/abnormal/boundary case indication
7. blackbox/whitebox test indication
8. functional/performance test indication
9. unit/integration test indication

Note that some of these categories may be inappropriate for your project and may be omitted if you can justify doing so. For items 6-9, only one term should apply.

III. *Test Case Matrix*: summarizes the test case coverage (items 1, 6-9 in a tabular format)

Example Test Plan Description

I. Overall Test Plan Strategies

We will use several different types of tests to ensure the accuracy and efficiency/performance of our Jigsaw Puzzle Solver (JPS). For the main tests, we will accumulate several testing puzzles with 10 to 100 pieces that are independent of the puzzles used for development. For each puzzle, we will perform normal, blackbox testing: use JPS to solve the puzzle and see if the program behaves as expected and has correct results (as specified by the design diagrams and specifications). To measure accuracy, we will need to manually put together the test puzzles (by hand), and ensure that the ordering of the pieces given by JPS matches what is seen in the physically completed puzzle (Test Case 1). We will perform functional tests to ensure that the user interface follows the technical specifications document, i.e., all menu options exist and work as specified, and the display of the completed puzzle is shown legibly (Case 2). We will be performing abnormal and boundary tests to make sure the program is reliable (Case 3). For efficiency and performance, we will have performance tests to make sure the program solves the test puzzles faster than humans can solve them (Case 4). Although the users of our program will have no knowledge of the separate components that comprise the complete algorithm, we will need to do unit and integration testing (whitebox testing) during development (Case 5).

Test Plan Specs

3.1 **Reliability.**
3.2 This test will ensure that the program can handle unexpected or abnormal inputs.
3.3 For the following items, verify the program (optionally) displays an error message but does not crash/close or stop responding.

- Choose the Open Dialog and attempt to choose a file that is not an image file.
- Choose the Open Dialog and choose an image file that does not contain puzzle pieces.
- Have JSP attempt to solve a puzzle that does not follow the assumptions: pieces having more than four neighbors, or not fully interlocking pieces.
- Have JSP attempt to solve a puzzle in which one or more pieces are missing.

3.4 Inputs: Abnormal user activities and abnormal puzzles
3.5 Outputs: Program may display an error message but will not crash or stop responding.
3.6 Abnormal
3.7 Blackbox
3.8 Functional
3.9 Integration (whole process)

III. Overall Test Case Matrix

Test Case ID	Normal/Ab.	Black/Whitebox	Funct/Perform	Unit/Integr.
1	Normal	Black	Function	Int.
2	Normal	Black	Function	Int.
3	Abnormal	Black	Function	Int.
4	Normal	Black	Performance	Int.
5	Normal	White	Function	Unit

UNIVERSITY OF Cincinnati